trace3d.py reference

Stage 1 of the 3D multipath simulator: scene description, Sionna RT path tracing and the 3D path file. Companion to channel3d.py, which turns the path file into channels. Path file format version 1.0. Tested with sionna-rt 2.2.0. Written 2026-10-24.

Contents

Overview 2

Conventions 2

1. Path file: schema and HDF5 I/O 3

PathSet3D . . . 3

PathSet3D.n_paths . . . 3

PathSet3D.rx_slice . . . 3

PathSet3D.delays . . . 3

PathSet3D.validate . . . 3

save_pathset3d . . . 4

load_pathset3d . . . 4

HDF5 layout . . . 4

2. Scene description and tracer 4

SceneSpec . . . 4

SceneSpec.valid . . . 5

SceneSpec.xml . . . 5

SceneSpec.to_dict . . . 5

make_rx_grid3d . . . 5

trace_paths3d . . . 5

What the tracer does, step by step . . . 6

Verification 6

Limitations and practical notes 7

Overview

What it does. `trace3d.py` finds the propagation paths between one transmitter and many receiver points inside a 3D room with box obstacles, and stores them in an HDF5 file. The path finding is done by Sionna RT. This module wraps it so that the scene, the antenna setup and the output format are fixed and reproducible.

Where it sits. Stage 1 is geometry plus materials at one frequency. Stage 2 (`channel3d.py`) applies the frequency band, the carrier-phase model and an optional receive array. Because Sionna evaluates reflection coefficients at the scene frequency, a path file is valid for one carrier: trace once per band.

Two parts. Section 1 of the file (the path-file schema and HDF5 reading and writing) has no dependency on Sionna, so the lightweight analysis environment can import it. Section 2 (scene description and tracer) imports Sionna and Mitsuba lazily inside the tracer, never at module level.

What is modelled.

Included	Not included (by design)
Line of sight; specular reflections up to a chosen order; occlusion by boxes; Fresnel reflection at each wall with the polarization carried through every bounce; free-space spreading; ITU-R P.2040 materials (via Sionna).	Diffraction; diffuse scattering; transmission through walls and boxes (so obstacles cast hard shadows); antenna patterns (one isotropic element at both ends); motion.

These exclusions must be stated alongside any result. Specular-only tracing gives sparser multipath than reality, and the effect grows with frequency.

Conventions

Quantity	Definition
Coordinates	Metres, right-handed. The room interior is [0,Lx] x [0,Ly] x [0,Lz]; z is up.
Path length L	Unfolded length of the path in metres. Delay is L / C0.
Amplitude a	Complex path amplitude WITHOUT the propagation phase: lambda / (4 pi L) times the (polarization-resolved) reflection factors, referenced to the isotropic vertically polarized antennas. The channel is H(f) = sum a exp(-j 2 pi f L / C0), the same convention as channel2d.
Order	Number of reflections; 0 is the line-of-sight path.
aoa	Unit vector at the receiver pointing toward where the ray comes from (back along the ray).
aod	Unit vector at the transmitter pointing along the departing ray.
fc_trace	Frequency in Hz at which the tracer evaluated the amplitudes.
rx_valid	False for receiver points outside the room or inside a box. Such receivers have no paths and become NaN rows downstream. A valid receiver with no path is a complete shadow and becomes a zero row.

Phase removal. Sionna's own path coefficients include the delay phase exp(-j 2 pi fc tau). The tracer removes it before saving, so the stored amplitude is real-positive for a line-of-sight path. This is what allows channel3d to either re-apply the geometric phase or replace it with a random carrier phase.

1. Path file: schema and HDF5 I/O

These items need only numpy and h5py.

`PathSet3D`

```
PathSet3D(tx, rx, rx_valid, offsets, length, order, a, aoa, aod, fc_trace, meta={})
```

Dataclass holding all paths from one transmitter to R receiver points. The per-path arrays are stored flat; the paths of receiver r are the rows offsets[r] to offsets[r+1].

Parameter	Type	Meaning
`tx`	(3,) float	Transmitter position (m).
`rx`	(R, 3) float	Receiver positions (m).
`rx_valid`	(R,) bool	False for receivers the tracer rejected.
`offsets`	(R+1,) int	Start index of each receiver's paths; the last entry equals the total number of paths P.
`length`	(P,) float	Unfolded path length (m).
`order`	(P,) int	Number of reflections (0 = line of sight).
`a`	(P,) complex	Amplitude without propagation phase.
`aoa`	(P, 3) float	Arrival unit vector at the receiver (points toward the source).
`aod`	(P, 3) float	Departure unit vector at the transmitter.
`fc_trace`	float	Trace frequency (Hz).
`meta`	dict	Free-form settings; the tracer records the scene, order, sample count, Mitsuba variant, Sionna version and the list of disabled effects.

`PathSet3D.n_paths`

```
n_paths() -> (R,) int
```

Number of paths of each receiver (the difference of offsets).

`PathSet3D.rx_slice`

```
rx_slice(r) -> slice
```

Slice that selects the paths of receiver r in any of the flat per-path arrays.

`PathSet3D.delays`

```
delays() -> (P,) float
```

Path delays in seconds (length / C0).

`PathSet3D.validate`

```
validate() -> None
```

Checks that the object is internally consistent and raises ValueError otherwise.

Notes. Checks: array shapes; offsets start at 0, never decrease and end at P; lengths finite and positive; aoa and aod are unit vectors (tolerance 1e-6); invalid receivers have no paths; fc_trace positive. It runs automatically on save and load.

save_pathset3d

```
save_pathset3d(filename, ps) -> None
```

Validates the PathSet3D and writes it to HDF5 with gzip compression (level 4) on the per-path arrays.

Parameter	Type	Meaning
filename	str or path	Output file; overwritten if it exists.
ps	PathSet3D	The path set to write.

load_pathset3d

```
load_pathset3d(filename) -> PathSet3D
```

Reads a file written by save_pathset3d and validates it.

Parameter	Type	Meaning
filename	str or path	Input file.

Notes. Raises ValueError if the file is not a 3D path file (for example a 2D path file or a channel file) or if the major format version differs from 1.

HDF5 layout

Location	Content
/tx/xyz	(3,) transmitter position
/rx/xyz, /rx/valid	(R, 3) receiver positions; (R,) validity flags
/offsets	(R+1,) path index ranges per receiver
/paths/length, order, a, aoa, aod	Flat per-path arrays (a is complex)
root attributes	format_version, kind = pathset3d, created_utc, fc_trace, meta_json

2. Scene description and tracer

The scene is deliberately restricted to a rectangular room containing axis-aligned boxes. This keeps the geometry exact, makes receiver validity a simple test, and lets the tracer be checked against analytic image-source results.

SceneSpec

```
SceneSpec(room, boxes=(), room_material='concrete', box_material='concrete',
          material_thickness=1.0)
```

Immutable description of the scene. The constructor raises ValueError if a room dimension is not positive or a box does not lie inside the room.

Parameter	Type	Meaning
room	(Lx, Ly, Lz)	Interior dimensions in metres.
boxes	tuple of 6-tuples	Solid obstacles (x0, y0, z0, x1, y1, z1) with lo < hi.
room_material	str	Material of the six room surfaces; a Sionna / ITU-R P.2040 name such as concrete, brick, plasterboard, wood, glass, metal.
box_material	str	Material of the obstacles.
material_thickness	float	Slab thickness (m) in Sionna's reflection model. A large value (default 1.0) behaves as a half-space and reproduces channel2d's Fresnel model; small values add slab interference.

Notes. The wall geometry itself is six slabs placed just outside the interior (0.1 m, physically irrelevant because transmission is off), so the interior faces tile the room exactly.

`SceneSpec.valid`

```
valid(pts, margin=1e-4) -> (R,) bool
```

Tests which points are usable receiver or transmitter positions: strictly inside the room and outside every box, keeping margin metres of clearance from all surfaces.

Parameter	Type	Meaning
`pts`	(R, 3) or (3,)	Points to test.
`margin`	float	Clearance in metres.

Notes. Points above or beside a box are valid; only the box volume is excluded.

`SceneSpec.xml`

```
xml() -> str
```

Returns the Mitsuba scene XML that Sionna loads: one ITU radio-material definition per distinct material, six wall slabs and one cube per box.

`SceneSpec.to_dict`

```
to_dict() -> dict
```

Plain-dictionary form of the specification, stored in the path file's metadata so every file records the scene it came from.

`make_rx_grid3d`

```
make_rx_grid3d(x_range, y_range, z, step) -> (points, xs, ys)
```

Builds a horizontal grid of receiver points at height z, ordered row by row (y in the outer loop, x in the inner loop), the same ordering as paths2d.make_rx_grid.

Parameter	Type	Meaning
`x_range, y_range`	(lo, hi)	Inclusive extents of the grid in metres.
`z`	float	Receiver height.
`step`	float	Grid spacing in metres.

Returns. points (R, 3) plus the 1D coordinate vectors xs and ys.

`trace_paths3d`

```
trace_paths3d(spec, tx, rx_points, fc, max_order=3, samples_per_src=1000000,
              batch=200, variant=None, progress=False, meta=None) -> PathSet3D
```

Traces line-of-sight and specular paths from the transmitter to every receiver point using Sionna RT, and returns them as a PathSet3D.

Parameter	Type	Meaning
`spec`	SceneSpec	The scene.
`tx`	(3,)	Transmitter position; must be inside the room and outside the boxes (ValueError otherwise).

Parameter	Type	Meaning
`rx_points`	(R, 3)	Receiver positions. Invalid ones are skipped and flagged.
`fc`	float	Carrier frequency in Hz at which materials are evaluated.
`max_order`	int	Maximum number of reflections (Sionna max_depth).
`samples_per_src`	int	Number of rays shot from the transmitter to discover specular paths. More rays find more paths at high orders but cost time.
`batch`	int	Receivers per Sionna call. The cost is dominated by the ray shooting, so larger batches are faster per receiver, at the price of memory.
`variant`	str or None	Mitsuba variant. None tries cuda_ad_mono_polarized (NVIDIA GPU) and falls back to llvm_ad_mono_polarized (CPU, needs LLVM).
`progress`	bool	Print coarse progress.
`meta`	dict or None	Extra metadata merged into the file's meta.

Returns. PathSet3D with paths sorted by length within each receiver, amplitudes de-phased as described under Conventions, and meta describing the run.

Notes. Fixed settings: one isotropic antenna with vertical polarization at both ends; line of sight and specular reflection on; diffuse reflection, refraction, diffraction and edge diffraction off. Sionna's refraction option defaults to on, so it is switched off explicitly.

What the tracer does, step by step

Step	Action
1	Check the transmitter; mark each receiver valid or invalid with SceneSpec.valid.
2	Import Sionna with the chosen Mitsuba variant, load the scene from SceneSpec.xml and set the scene frequency to fc.
3	Place the transmitter and, per batch, the valid receivers. Run Sionna's path solver with the fixed settings above.
4	From the result take the complex amplitudes, delays, validity flags, interaction counts (giving the order) and the arrival and departure angles.
5	Per receiver keep valid paths, sort by delay, convert the angles to unit vectors, convert delay to length and remove the delay phase from the amplitude.
6	Assemble the flat arrays and offsets into a PathSet3D.

Internal helper. `_import_sionna(variant)` sets the Mitsuba variant and then imports sionna.rt. With no variant it first tries the CUDA variant, forces the GPU driver to load, and falls back to the CPU variant on any failure. It is private, but is the place to look if the GPU is not being used.

Verification

The tests are in `test_trace3d.py`. The tracer tests are skipped automatically when sionna-rt is not installed, and the environment variable TRACE3D_VARIANT forces a Mitsuba variant.

Check	Result
Empty box, orders 1 to 3, three receivers	Path counts and lengths equal the image-source lattice (agreement about 1e-6 m).
Line-of-sight amplitude	Equals lambda / (4 pi L); real positive after phase removal; aoa and aod are the expected unit vectors.

Check	Result
Single reflections off all six surfaces	Amplitude equals lambda / (4 pi L) times the Fresnel TE and TM coefficients (channel2d.fresnel) applied to the vertical polarization vector, to within 1e-3. Floor and ceiling are mostly TM and the side walls mostly TE, with a small mix because rays are tilted.
Obstacles	A full-height box removes the line-of-sight path behind it; receivers inside the box are flagged invalid with no paths.
Batching	Batch sizes 1 and 100 give the same paths.
Pipeline	A traced file round-trips through HDF5 and builds finite channels in both channel3d phase modes.

Limitations and practical notes

Topic	Note
Completeness	Sionna finds specular paths by shooting rays, so discovery is statistical. It matched the analytic lattice exactly up to order 3 in the test box, but at order 4 one of 129 paths was missed even with 10 million rays. Check completeness for your own scene and sample count before relying on high orders.
One trace per band	Materials are evaluated at the scene frequency. channel3d refuses a band whose carrier differs from fc_trace unless told otherwise.
Material thickness	The default of 1.0 m makes walls behave as half-spaces. Thin slabs change reflection amplitudes by a few percent and add frequency dependence.
Precision	Sionna works in single precision. Delays are converted to double precision afterwards; the resulting phase error is about 1e-3 rad for paths of a few hundred metres, which is irrelevant in random-carrier mode.
Hard shadows	With transmission, diffraction and scattering off, receivers behind a box see only reflected paths, so shadows are sharper and deeper than in reality.
Cost	The run time is dominated by samples_per_src, and grows with the number of batches. The first call is slow because of kernel compilation.
Invalid points	A transmitter that is outside the room or in a box raises an error; invalid receivers are silently flagged, so always mask with rx_valid downstream.
Environment	Needs sionna-rt (which brings Mitsuba and Dr.Jit). Without a working NVIDIA driver, install LLVM or pass the CPU variant. The path-file I/O does not need Sionna.